

StreamingApp — DevOps Implementation

Issues Log & Resolutions

Docker → Kubernetes → Helm → Terraform → Ansible → Jenkins → AWS (EKS/ECR/S3/ALB) → Teardown

Covers the full run from IAM user creation through final AWS resource cleanup.

Prepared as a project record of problems encountered, their root causes, and how each was resolved.

Phases Covered

1. Local Validation — Docker & Compose
2. Local Validation — Kubernetes Manifests
3. Local Validation — Helm Chart
4. Local Validation — Terraform Structure
5. Git Hygiene
6. AWS Credentials & CLI Setup
7. Terraform — AWS Infrastructure Provisioning
8. Ansible — Jenkins Server Configuration
9. Docker Images — Build & Push to ECR
10. Application Deployment — Helm to EKS
11. Persistent Storage — MongoDB / EBS CSI
12. Networking — ALB Ingress
13. Jenkins Pipeline
14. Verification
15. Teardown / Cleanup

Phase 1 — Local Validation: Docker & Docker Compose

1. COPY path mismatch in adminService / chatService Dockerfiles

SYMPTOM Docker build failed — could not find `package.json` / source files inside build context.

ROOT CAUSE These Dockerfiles used a build context of `./backend` (the whole backend folder) but had `COPY package*.json ./` without the service-name prefix, so Docker looked for the files at the context root instead of inside the service subfolder.

FIX Prefixed every `COPY` instruction with the service folder name (e.g. `COPY adminService/package*.json ./` , `COPY chatService/. ./`) to match streamingService's already-working pattern.

2. adminService container crashed with EACCES on /app/tmp-uploads

SYMPTOM Container exited immediately with a permission-denied error writing to a temp upload directory.

ROOT CAUSE The Dockerfile switched to a non-root `appuser` before the working directory's ownership was changed, so the app couldn't write to `/app` .

FIX Added `RUN chown -R appuser:appgroup /app` immediately before the `USER appuser` instruction.

3. Frontend nginx: "host not found in upstream auth-service"

SYMPTOM nginx failed to start inside the frontend container with a DNS resolution error for backend hostnames.

ROOT CAUSE docker-compose's default bridge network only auto-registers a container's service name (`auth` , not `auth-service`), but nginx.conf referenced the longer hostnames used elsewhere.

FIX Added explicit `networks.default.aliases` under each backend service in `docker-compose.yml` (e.g. `aliases: [auth-service]`) so both names resolve.

4. Frontend HEALTHCHECK failing despite nginx running fine

SYMPTOM Docker reported the frontend container as `unhealthy` even though it served pages correctly.

ROOT CAUSE `HEALTHCHECK ... wget http://localhost/health` resolved `localhost` to the IPv6 loopback (`::1`) inside the container, but nginx only listens on IPv4.

FIX Changed the healthcheck URL to the explicit IPv4 loopback: `http://127.0.0.1/health`.

Phase 2–4 — Local Validation: Kubernetes / Helm / Terraform Structure

5. Widespread YAML corruption across k8s manifests and Helm templates

SYMPTOM `kubect1 apply / helm template / helm lint` failed with indentation and parsing errors across nearly every manifest.

ROOT CAUSE The original project files were copy-pasted out of a PDF plan document, which flattened indentation, glued together document separators (`---` became `--`), and broke nested keys like `envFrom / secretRef`.

FIX Rewrote each affected file (k8s Deployments/Services/StatefulSet/HPA/Ingress, Helm templates and values.yaml) with correct YAML structure, verified via `kubect1 apply --dry-run=client` and `helm template` against a local `kind` cluster.

6. Helm values.yaml key casing violated Kubernetes naming rules

SYMPTOM `helm install` rejected generated object names (RFC 1123 subdomain violation).

ROOT CAUSE The original `values.yaml` used camelCase keys (e.g. `authService`) that were used directly as Kubernetes object names, but K8s names must be lowercase and hyphen-separated.

FIX Restructured `values.yaml` to use a `services:` map with kebab-case keys (`auth-service`, `streaming-service`, etc.), and templates iterate with `{{ range $svcName, $svc := .Values.services }}`, using `$svcName` directly as the object name.

Phase 5 — Git Hygiene

7. GitHub Personal Access Token exposed in git remote URL **[CRITICAL]**

SYMPTOM Running `git remote -v` revealed a PAT embedded directly in the HTTPS remote URL.

ROOT CAUSE The remote had been configured at some point using the pattern `https://<token>@github.com/...` instead of a credential helper.

FIX Advised immediate revocation of the token on GitHub, then reset the remote URL to the plain HTTPS form without embedded credentials, relying on the OS credential manager for auth.

8. 685 MB Terraform provider binary and a video file committed to git

SYMPTOM Repository size ballooned; a push would have been slow/rejected.

ROOT CAUSE `.terraform/` provider plugin binaries and a test video asset were committed because `.gitignore` had no Terraform-specific rules.

FIX Added a Terraform section to `.gitignore` (`/**/*.terraform/`, `*.tfstate*`, `*.tfplan`, `*.tfvars`, etc. — deliberately keeping `.terraform.lock.hcl` tracked), then ran `git reset HEAD~1` to undo the offending commit (safe since it was unpushed).

Phase 6 — AWS Credentials & CLI Setup

9. Placeholder text committed into AWS credentials files

SYMPTOM Every AWS CLI call failed with an authentication error.

ROOT CAUSE The literal placeholder string (e.g. `<your streamingapp-terraform access key>`) had been pasted into both the Windows (`$HOME\.aws\credentials`) and WSL (`~/.aws/credentials`) files instead of the real generated key values.

FIX Replaced placeholders with the actual IAM user access key/secret in both locations.

10. AWS_PROFILE not persisting / named profile confusion

SYMPTOM Commands worked in one terminal session and failed with credential errors in a fresh one.

ROOT CAUSE A named profile (`streamingapp`) required `AWS_PROFILE` to be set per-session; it wasn't being persisted, and Windows vs. WSL credential files were also being mixed up.

FIX Simplified by moving all credentials to the `[default]` profile on both sides, removing the need to set `AWS_PROFILE` at all.

Phase 7 — Terraform: AWS Infrastructure Provisioning

11. EC2 instance type blocked by Free Tier

SYMPTOM `terraform apply` failed with `InvalidParameterCombination` when provisioning the Jenkins EC2 instance as `t3.medium`.

ROOT CAUSE The AWS account's Free Tier / service-quota configuration only permits micro/small instance types.

FIX Confirmed eligible types via `aws ec2 describe-instance-types` and switched `node_instance` / the Jenkins instance to `t3.small` in `variables.tf` / `ec2.tf`.

12. Unsupported EKS Kubernetes version

SYMPTOM `InvalidParameterException: unsupported Kubernetes version` when creating the EKS cluster on version 1.29.

ROOT CAUSE EKS 1.29 had aged out of AWS's standard support window by the time of provisioning.

FIX Checked currently supported versions via `aws eks describe-cluster-versions` and bumped `cluster_version` to 1.33.

13. "Already exists" errors on KMS alias and EC2 key pair during re-apply

SYMPTOM A fresh `terraform apply` after a prior partial/manual cleanup failed, claiming the KMS alias and key pair already existed.

ROOT CAUSE An earlier incomplete manual cleanup had left orphaned resources behind: the EKS-auto-created KMS key was in `PendingDeletion` but its alias still existed and pointed at it.

FIX Ran `aws kms delete-alias` to remove the stale alias, and confirmed via `describe-key-pairs` that the key pair conflict was a naming collision rather than a real duplicate before re-applying.

14. SSH "Permission denied (publickey)" to the Jenkins EC2 instance [CRITICAL]

SYMPTOM SSH connection to the Jenkins instance was rejected even though the local `.pub` file appeared to match what was registered in AWS.

ROOT CAUSE A raw byte-level comparison of `aws ec2 describe-key-pairs --include-public-key` output against the local `.pub` file proved they were genuinely different keys — AWS held a stray, manually created key pair under the same name from an earlier attempt.

FIX Forced replacement of both resources with `terraform apply -replace="aws_key_pair.streamingapp" -replace="aws_instance.jenkins"`, which regenerated the key pair from the correct local public key and recreated the instance bound to it.

15. terraform.exe intermittently blocked by Windows Application Control policy

SYMPTOM Recurring, unpredictable failures launching `terraform.exe` for specific subcommands (notably `terraform output` / `terraform plan`).

ROOT CAUSE Never fully root-caused — appears to be a Windows endpoint security / Application Control policy intermittently flagging the binary.

FIX Worked around case-by-case by querying the same information directly via AWS CLI instead of the blocked Terraform subcommand.

Phase 8 — Ansible: Jenkins Server Configuration

16. Dynamic inventory indentation errors

SYMPTOM `ansible-playbook` failed to parse `ansible/inventory/aws_ec2.yml` .

ROOT CAUSE The `compose:` block (needed to map `ansible_host` / `ansible_user` / `ansible_ssh_private_key_file` to the EC2 dynamic inventory) had incorrect indentation, recurring across multiple edits.

FIX Corrected the block to be a sibling of `keyed_groups` / `hostnames` at the top level, each key properly quoted where it held a literal string expression.

17. "unzip: not found" during AWS CLI install

SYMPTOM The `kubectl` role's shell task to install AWS CLI v2 failed because `unzip` wasn't available on the base Ubuntu image.

ROOT CAUSE `unzip` is not installed by default on the Ubuntu AMI used, and no task installed it before it was needed.

FIX Added an `apt: name: unzip` task before the AWS CLI install step in `ansible/roles/kubectl/tasks/main.yml` .

18. Docker GPG key installation failing (`apt_key` deprecated)

SYMPTOM Ansible's `apt_key` module task failed / warned strongly on Ubuntu 22.04.

ROOT CAUSE Ubuntu 22.04 deprecated the legacy `apt-key` mechanism in favor of per-repository signed-by keyrings.

FIX Diagnosed via `WebFetch` that Jenkins' own signing key had also rotated; switched both Docker's and Jenkins' key installation to the modern pattern — download the key with `get_url` into `/etc/apt/keyrings/` , then reference it with `signed-by=` in the `apt_repository` task (Jenkins key URL updated to `jenkins.io-2026.key`).

19. Jenkins service failed to start — Java version too old **[CRITICAL]**

SYMPTOM `systemctl status jenkins` showed the service repeatedly failing; no web UI reachable on port 8080.

ROOT CAUSE Diagnosed by manually running `sudo -u jenkins /usr/bin/jenkins` on the instance, which surfaced: `Running with Java 17 ... older than minimum required version (Java 21)` . Current Jenkins LTS requires Java 21, but the role had installed Java 17.

FIX Changed the Ansible task to install `openjdk-21-jre` and added an explicit `alternatives` task to set Java 21 as the default `java` binary system-wide.

Phase 9 — Docker Images: Build & Push to ECR

20. ECR docker login failing with 400 Bad Request via PowerShell

SYMPTOM Piping the ECR login password directly into `docker login --password-stdin` from PowerShell returned an HTTP 400 from the registry.

ROOT CAUSE Root-caused to PowerShell's native-executable pipe handling corrupting the password's encoding when passed through a pipe (`aws ecr get-login-password | docker login ...`).

FIX Captured the password into a PowerShell variable first, then passed it directly as an argument: `docker login --password $loginPassword` , avoiding the native pipe entirely.

Phase 10 — Application Deployment: Helm to EKS

21. All pods stuck in ImagePullBackOff / InvalidImageName **[CRITICAL]**

SYMPTOM Every service pod failed to start after `helm upgrade --install` , reporting an invalid image reference.

ROOT CAUSE `values.yaml` 's `global.imageRegistry` still held the literal unsubstituted placeholder `<AWS_ACCOUNT_ID>.dkr.ecr...` — it had never been templated or overridden.

FIX Overrode it at deploy time rather than editing the file: `--set global.imageRegistry=024757002386.dkr.ecr.us-east-1.amazonaws.com` on the `helm upgrade` command line.

22. Missing EKS access entry — kubectl/console access denied for the creating IAM identity

SYMPTOM `kubectl` commands against the new cluster returned an authorization error even though the cluster had just been created.

ROOT CAUSE EKS access entries must be granted explicitly per IAM principal — the cluster creator does not get automatic access unless `enable_cluster_creator_admin_permissions = true` is set on the `terraform-aws-modules/eks/aws` module, which it wasn't initially.

FIX Granted an explicit EKS access entry / enabled creator admin permissions so the working IAM user could authenticate against the cluster.

Phase 11 — Persistent Storage: MongoDB / EBS CSI

23. Mongo PVC stuck in Pending — FailedScheduling [CRITICAL]

SYMPTOM The `mongo-data` `PersistentVolumeClaim` never bound; the Mongo pod stayed unscheduled with unbound `PersistentVolumeClaims`.

ROOT CAUSE The EKS cluster had no `aws-ebs-csi-driver` add-on installed, and the legacy in-tree `kubernetes.io/aws-ebs` provisioner is no longer functional on current EKS versions — there was no working dynamic volume provisioner.

FIX Created an IRSA (IAM Roles for Service Accounts) role for the CSI driver, installed the add-on via `aws eks create-addon`, created a new `gp3-csi` `StorageClass` set as default, then deleted and recreated the stuck PVC/pod so it would bind against the new provisioner.

Phase 12 — Networking: ALB Ingress

24. ALB target groups reporting `Target.ResponseCodeMismatch` on all 4 backend services [CRITICAL]

SYMPTOM The AWS Load Balancer Controller provisioned target groups, but every backend service's targets were marked unhealthy.

ROOT CAUSE A single Ingress object can only carry one global `alb.ingress.kubernetes.io/healthcheck-path` annotation. The original Ingress set it to `/health`, which is correct for the frontend but wrong for the 4 API services, whose real health route is `/api/health`.

FIX Split into two Ingress objects sharing one ALB via `alb.ingress.kubernetes.io/group.name: streamingapp` — `ingress-frontend.yaml` keeps `healthcheck-path: /health`, and a new `ingress-api.yaml` uses `healthcheck-path: /api/health` for the auth/streaming/admin/chat routes.

25. Two ALBs running simultaneously after the Ingress split

SYMPTOM AWS console showed two Application Load Balancers for the app instead of one, doubling cost.

ROOT CAUSE The old single `streamingapp-ingress` object was never actually deleted before the new split Ingress objects were applied.

FIX Deleted the old object explicitly by name — `kubectl delete ingress streamingapp-ingress -n streamingapp` — rather than relying on a stale file reference with `-f`.

26. Frontend still returning 503 after the target-group fix [CRITICAL]

SYMPTOM API routes worked through the ALB, but the root path serving the frontend still returned a 503.

ROOT CAUSE `ingress-frontend.yaml` 's backend referenced a Service named `frontend-service`, but the Helm chart's `service.yaml` template creates the Service literally named `frontend` (from the `services:` map key) — that Service never existed.

FIX Corrected the Ingress backend's `service.name` field to `frontend`.

Phase 13 — Jenkins Pipeline

27. Docker build stage using wrong context/Dockerfile pairs

SYMPTOM The Jenkinsfile's Docker Build & Push stage failed for several services with "file not found" during build.

ROOT CAUSE The service list in the pipeline used one uniform context/Dockerfile convention, but the actual Dockerfiles were inconsistent about whether they expected the build context at the service folder or the shared `backend` folder (see Issue 1).

FIX Set each service's `context / dockerfile` pair in the Jenkinsfile's `services` list to match its actual Dockerfile's COPY convention.

28. Unit Test stage always failing

SYMPTOM The pipeline's Unit Tests stage failed on every run, even with no actual test code.

ROOT CAUSE npm's default generated `"test"` script (`exit 1`, "no test specified") is designed to fail by default until real tests are added.

FIX Changed the placeholder script in `authService / streamingService package.json` to `"echo \"no tests specified\" && exit 0"` as an interim measure pending real test coverage.

29. Smoke Tests stage failing against wrong URLs

SYMPTOM The post-deploy Smoke Tests stage always failed, even when the app was demonstrably reachable through the ALB.

ROOT CAUSE The stage curled external ALB-prefixed paths that didn't match the actual Ingress path routing, and one service's health route was inconsistent (`/health` vs `/api/health`).

FIX Rewrote the stage to run internal `kubectl run curl` checks directly against each service's ClusterIP DNS name and `/api/health` path, and standardized `authService/index.js` to mount its health route at `/api/health` for consistency with the other three services.

30. Deploy/Smoke stages running on every branch

SYMPTOM Feature-branch builds attempted to apply Terraform and deploy to production EKS.

ROOT CAUSE The Terraform Apply, Ansible Configure, Deploy to EKS, and Smoke Tests stages had no branch guard.

FIX Added `when { branch 'main' }` to each of those stages so only merges to `main` trigger real infrastructure changes.

Phase 15 — Teardown / Cleanup

31. Terraform state lost mid-teardown — S3 state bucket deleted before terraform destroy **[CRITICAL]**

SYMPTOM During the cost-prioritized cleanup, `terraform state list` began failing with `NoSuchBucket`. All live, still-billing AWS resources (EKS cluster, node group, EC2, NAT Gateway, etc.) instantly became untracked by Terraform.

ROOT CAUSE The S3 bucket holding the Terraform state (`streamingapp-tfstate`) was deleted out of order — before `terraform destroy` had run — removing Terraform's only record of what it was managing while the resources were still live.

FIX Abandoned the Terraform-driven teardown and switched to manual, dependency-ordered deletion directly via AWS CLI: EKS node group → EKS cluster → Jenkins EC2 instance → NAT Gateway + Elastic IP, each step verified independently with `describe-*` calls before moving to the next, given the higher stakes of an untracked cleanup.

32. S3 bucket deletion failing with `BucketNotEmpty` despite the bucket "looking" empty

SYMPTOM `aws s3 rb ... --force` failed on the tfstate bucket with `BucketNotEmpty`.

ROOT CAUSE The bucket had versioning enabled; regular object deletion leaves historical versions and delete-markers behind, which still count as bucket contents.

FIX Enumerated and explicitly deleted every object version and delete-marker via `aws s3api list-object-versions` + `delete-object --version-id` before removing the bucket itself. Applied the same pattern later for the `streamingapp-videos` bucket.

33. Security groups blocking VPC deletion — DependencyViolation

SYMPTOM Deleting the EKS-created cluster and node security groups, and later the VPC itself, failed with `DependencyViolation` even though no EC2 instances or ENIs remained.

ROOT CAUSE The two EKS-managed security groups referenced each other in their ingress rules (cluster→node and node→node/cluster for kubelet, webhook, and CoreDNS ports) — AWS won't delete a security group that's still referenced by another group's rule, regardless of whether any instance uses it.

FIX Enumerated every ingress rule ID on both groups via `describe-security-group-rules`, revoked all cross-referencing rules with `revoke-security-group-ingress`, then deleted both groups and finally the VPC cleanly.

34. Orphaned EC2 key pair left behind after full teardown

SYMPTOM After all billable resources were confirmed deleted, the `streamingapp-key` EC2 key pair still existed in the account.

ROOT CAUSE Key pairs aren't part of the VPC/EKS/EC2 dependency chain, so they were skipped by the dependency-ordered deletion sequence used for the rest of teardown.

FIX Explicitly deleted via `aws ec2 delete-key-pair --key-name streamingapp-key` once flagged.